

Complete Notes on Unit 3 and Unit 4

Heap Sort, Non-Comparison Sorts, Trees, Graphs, Hashing, Tries, Bloom Filters, Spatial Structures, Fenwick/Segment Trees, and Skip Lists

How to Study These Notes

These notes are designed to help students in four ways:

1. **Understand the idea** behind each topic.
2. **See the working process** step by step.
3. **Connect the topic to real life** and industry.
4. **Implement the topic in Python** using the heavily commented programs.

For each topic, first read the concept, then the example, then the algorithm, and finally the code.

Contents

1	Heap Sort	4
1.1	Why Heap Sort Matters	4
1.2	Core Idea	4
1.3	Complexity	6
1.4	Implementation Steps	6
1.5	Simple Python Program	6
2	Counting Sort	7
2.1	When Counting Sort Works Best	7
2.2	Main Idea	8
2.3	Why Traverse from Right to Left?	8
2.4	Complexity	9
2.5	Implementation Steps	9
2.6	Simple Python Program	9
3	Radix Sort	10
3.1	Main Idea	10
3.2	Why Stability Is Important	11
3.3	Complexity	11
3.4	Implementation Steps	11
3.5	Simple Python Program	12
4	Trees: Terminology, Binary Tree Types, and Traversals	13
4.1	Important Terminology	13
4.2	Binary Tree	13
4.3	Types of Binary Trees	13
4.4	Tree Traversals	14
4.4.1	Depth-First Traversals	14
4.4.2	Breadth-First Traversal	14
4.5	Implementation Steps	15
4.6	Simple Python Program	15

5	Binary Search Trees (BSTs): Properties and Core Operations	17
5.1	BST Property	17
5.2	Why BST Is Useful	17
5.3	Core Operations	17
5.4	Complexity	18
5.5	Implementation Steps	18
5.6	Simple Python Program	18
6	Heaps: Min/Max Heap Properties, Heapify, and Operations	20
6.1	Why Heap Is Special	20
6.2	Array Representation	20
6.3	Main Operations	20
6.4	Implementation Steps	21
6.5	Simple Python Program	21
7	Balanced Trees (Conceptual): AVL Trees, Red-Black Trees, and B-Trees	23
7.1	Why Balancing Is Needed	23
7.2	AVL Tree	23
7.2.1	AVL Rotations	24
7.3	Red-Black Tree	24
7.4	B-Tree	24
7.5	Step-by-Step AVL Insertion Example	25
7.6	Implementation Steps	25
7.7	Simple Python Program (AVL Tree Insertion)	25
8	Graphs: Terminology and Representations	27
8.1	Terminology	27
8.2	Graph Representations	27
8.2.1	Adjacency Matrix	27
8.2.2	Adjacency List	28
8.3	Implementation Steps	28
8.4	Simple Python Program	29
9	Graph Traversal: BFS and DFS	29
9.1	BFS	29
9.2	DFS	29
9.3	Applications	30
9.4	Simple Python Program	31
10	Hashing: Concept, Hash Function, and Collision Resolution	32
10.1	Main Idea	32
10.2	Hash Table	32
10.3	What Makes a Good Hash Function?	32
10.4	Collision	33
10.5	Collision Resolution Techniques	33
10.6	Complexity	33
10.7	Simple Python Program	33
11	Tries (Prefix Trees): Concept and Applications	35
11.1	Main Idea	35
11.2	Structure	35
11.3	Operations	35
11.4	Implementation Steps	36
11.5	Simple Python Program	36

12 Bloom Filters	37
12.1 Main Idea	37
12.2 Implementation Steps	38
12.3 Simple Python Program	39
13 Spatial Data Structures: Quadtrees and K-D Trees	40
13.1 Why Spatial Structures Are Needed	40
13.2 Quadtrees	40
13.3 K-D Trees	40
13.4 Implementation Steps for K-D Tree	41
13.5 Simple Python Program (K-D Tree)	41
13.6 Mini Python Program (Quadtree Conceptual Skeleton)	42
14 Fenwick Tree and Segment Tree	43
14.1 Fenwick Tree (Binary Indexed Tree)	43
14.1.1 Main Idea	43
14.2 Segment Tree	43
14.3 Fenwick Tree Implementation Steps	44
14.4 Simple Python Program (Fenwick Tree)	44
14.5 Simple Python Program (Segment Tree)	45
15 Skip Lists	46
15.1 Main Idea	46
15.2 Why Skip Lists Are Interesting	47
15.3 Implementation Steps	47
15.4 Simple Python Program	47
16 Quick Revision Tables	49
16.1 Sorting Summary	49
16.2 Tree and Graph Summary	49
17 Common Mistakes Students Must Avoid	50
18 Practice Questions for Students	50

1. Heap Sort

Concept

Heap Sort is a comparison-based sorting algorithm that uses the **heap data structure**. A heap is a complete binary tree that satisfies the heap property:

- In a **max-heap**, every parent is greater than or equal to its children.
- In a **min-heap**, every parent is smaller than or equal to its children.

Heap Sort usually uses a **max-heap** to sort in ascending order.

1.1 Why Heap Sort Matters

Heap Sort is important because:

- It guarantees $O(n \log n)$ time in the worst case.
- It sorts **in-place**, so it uses very little extra memory.
- It is useful in systems where predictable performance matters.

1.2 Core Idea

If we build a **max-heap**, the largest element comes to the root. Then:

1. Swap the root with the last element.
2. Reduce the heap size by one.
3. Restore the heap property using **heapify**.
4. Repeat until the list becomes sorted.

Worked Example

Example: Sort the array [4, 10, 3, 5, 1]

Step 1: Build max-heap After heap construction, the array becomes:

[10, 5, 3, 4, 1]

Step 2: Swap root and last

[1, 5, 3, 4, 10]

Now heap size becomes 4.

Step 3: Heapify first 4 elements

[5, 4, 3, 1, 10]

Step 4: Swap root and last of current heap

[1, 4, 3, 5, 10]

Heap size becomes 3.

Step 5: Heapify first 3 elements

[4, 1, 3, 5, 10]

Step 6: Repeat Final sorted array:

[1, 3, 4, 5, 10]

Algorithm / Working Steps**Heap Sort Algorithm**

1. Convert the input array into a max-heap.
2. Let the heap size be the full size of the array.
3. Swap the first element (largest) with the last element of the current heap.
4. Reduce heap size by 1.
5. Call heapify on the root to restore max-heap property.
6. Repeat until heap size becomes 1.

Heapify Process

1. Compare the parent with left child.
2. Compare the current largest with right child.
3. If the parent is not the largest, swap with the largest child.
4. Recursively heapify the affected subtree.

1.3 Complexity

Time / Space	Complexity
Build Heap	$O(n)$
Heapify (single call)	$O(\log n)$
Heap Sort Total	$O(n \log n)$
Extra Space	$O(1)$
Stable?	No

Daily-Life / Industry Use

Daily-life analogy: Imagine arranging students by height for a school parade. If the tallest student is always quickly identified and moved to the final position, and then the next tallest is found similarly, the arrangement process resembles Heap Sort.

Industry use:

- Priority-based job scheduling
- Event simulation
- Resource allocation
- Real-time systems where worst-case performance matters

1.4 Implementation Steps

1. Store elements in an array.
2. Treat the array like a complete binary tree:

$$\text{left child} = 2i + 1, \quad \text{right child} = 2i + 2$$

3. Build the max-heap from the last non-leaf node to the root.
4. Swap the root with the last unsorted element.
5. Heapify the reduced heap.

1.5 Simple Python Program

```

1 # heapify() fixes the heap property for a subtree rooted at index i.
2 def heapify(arr, n, i):
3     # Assume the current index i stores the largest value for now.
4     largest = i
5
6     # Compute the index of the left child of node i.
7     left = 2 * i + 1
8
9     # Compute the index of the right child of node i.
10    right = 2 * i + 2
11
12    # Check whether the left child exists inside the heap
13    # and whether its value is larger than the current largest value.
14    if left < n and arr[left] > arr[largest]:
15        largest = left

```

```
16
17     # Check whether the right child exists inside the heap
18     # and whether its value is larger than the current largest value.
19     if right < n and arr[right] > arr[largest]:
20         largest = right
21
22     # If the largest value is not at index i,
23     # then swap the parent with the larger child.
24     if largest != i:
25         arr[i], arr[largest] = arr[largest], arr[i]
26
27         # After swapping, the affected subtree may violate heap property,
28         # so recursively fix that subtree.
29         heapify(arr, n, largest)
30
31
32 # heap_sort() sorts the list arr in ascending order.
33 def heap_sort(arr):
34     # Find the total number of elements in the array.
35     n = len(arr)
36
37     # Build a max-heap.
38     # Start from the last non-leaf node and move upward to the root.
39     for i in range(n // 2 - 1, -1, -1):
40         heapify(arr, n, i)
41
42     # One by one move the current largest element to the end.
43     for i in range(n - 1, 0, -1):
44         # Swap the root (largest value) with the last element
45         # in the current unsorted portion.
46         arr[i], arr[0] = arr[0], arr[i]
47
48         # Restore the heap property for the reduced heap.
49         heapify(arr, i, 0)
50
51
52 # Example input list.
53 numbers = [4, 10, 3, 5, 1]
54
55 # Sort the list in-place.
56 heap_sort(numbers)
57
58 # Print the final sorted result.
59 print("Sorted array:", numbers)
```

Listing 1: Heap Sort in Python with Detailed Comments

2. Counting Sort

Concept

Counting Sort is a **non-comparison sorting algorithm**. It does not compare one element with another like Bubble Sort or Heap Sort. Instead, it counts how many times each value appears, and then places the values in order.

2.1 When Counting Sort Works Best

Counting Sort works best when:

- The values are **integers**.
- The range of values is not very large.
- Example: numbers from 0 to 100.

2.2 Main Idea

If the input is:

[4, 2, 2, 8, 3, 3, 1]

then we count the frequency of every number:

$1 \rightarrow 1, \quad 2 \rightarrow 2, \quad 3 \rightarrow 2, \quad 4 \rightarrow 1, \quad 8 \rightarrow 1$

Then we use these counts to reconstruct the sorted output.

Worked Example

Example: Sort [4, 2, 2, 8, 3, 3, 1]

Step 1: Find maximum value Maximum = 8

Step 2: Create count array Indices represent values from 0 to 8

count = [0, 1, 2, 2, 1, 0, 0, 0, 1]

Step 3: Convert count to prefix sums

[0, 1, 3, 5, 6, 6, 6, 6, 7]

Step 4: Place elements in output array The prefix sums tell the final position of each element.

Final sorted array

[1, 2, 2, 3, 3, 4, 8]

Algorithm / Working Steps

Counting Sort Algorithm

1. Find the maximum element.
2. Create a count array of size $\text{max} + 1$.
3. Count the occurrences of each element.
4. Convert count array into cumulative count array.
5. Traverse the original array from right to left.
6. Place each element into its correct position in the output array.
7. Copy the output back if needed.

2.3 Why Traverse from Right to Left?

Traversing from right to left makes Counting Sort **stable**. A stable sort keeps equal elements in the same relative order as the original array.

2.4 Complexity

$$O(n + k)$$

where:

- n = number of elements
- k = range of values

Measure	Complexity
Time	$O(n + k)$
Space	$O(n + k)$
Stable?	Yes
Comparison-based?	No

Daily-Life / Industry Use

Daily-life analogy: Suppose exam marks are out of 10. Instead of comparing every student's marks with every other student, we just count how many students got 0, 1, 2, ..., 10. Then we list them in order.

Industry use:

- Sorting small-range integer data
- Histogram building in image processing
- Age-group distribution analysis
- Ranking systems where value range is limited

2.5 Implementation Steps

1. Read all integers.
2. Determine the largest value.
3. Initialize a count array.
4. Count each value.
5. Convert counts to positions.
6. Build the sorted output.

2.6 Simple Python Program

```

1 # counting_sort() returns a new sorted list.
2 def counting_sort(arr):
3     # If the input list is empty, return it directly.
4     if not arr:
5         return arr
6
7     # Find the maximum value in the list.
8     max_value = max(arr)
9
10    # Create a count array where each index represents a number,
```

```
11     # and the value stored there represents how many times
12     # that number appears in the input list.
13     count = [0] * (max_value + 1)
14
15     # Count the frequency of each number.
16     for number in arr:
17         count[number] += 1
18
19     # Convert the frequency array into a prefix-sum array.
20     # After this step, count[i] tells the final position
21     # of the value i in the sorted output.
22     for i in range(1, len(count)):
23         count[i] += count[i - 1]
24
25     # Create the output array of the same size as the input.
26     output = [0] * len(arr)
27
28     # Traverse the input from right to left to keep the sort stable.
29     for i in range(len(arr) - 1, -1, -1):
30         number = arr[i]
31         output[count[number] - 1] = number
32         count[number] -= 1
33
34     # Return the sorted output list.
35     return output
36
37
38 # Example input list.
39 numbers = [4, 2, 2, 8, 3, 3, 1]
40
41 # Call the sorting function and store the result.
42 sorted_numbers = counting_sort(numbers)
43
44 # Display the final sorted list.
45 print("Sorted array:", sorted_numbers)
```

Listing 2: Counting Sort in Python with Detailed Comments

3. Radix Sort

Concept

Radix Sort is also a **non-comparison sorting algorithm**. It sorts numbers digit by digit. Usually, it sorts from the **least significant digit (LSD)** to the most significant digit (MSD).

3.1 Main Idea

For the list:

[170, 45, 75, 90, 802, 24, 2, 66]

we sort by:

1. Units place
2. Tens place
3. Hundreds place

At each step we use a **stable sort** such as Counting Sort.

Worked Example

Example: Sort [170, 45, 75, 90, 802, 24, 2, 66]

Pass 1: Units digit After sorting by units digit:

[170, 90, 802, 2, 24, 45, 75, 66]

Pass 2: Tens digit

[802, 2, 24, 45, 66, 170, 75, 90]

Pass 3: Hundreds digit

[2, 24, 45, 66, 75, 90, 170, 802]

Algorithm / Working Steps

Radix Sort Algorithm

1. Find the maximum number.
2. Start with the least significant digit (units place).
3. Apply a stable sort based on the current digit.
4. Move to the next digit place: tens, hundreds, thousands, etc.
5. Continue until the highest digit place of the maximum number is processed.

3.2 Why Stability Is Important

If we sort by units digit first, and then by tens digit, the previous order among equal tens digits must remain unchanged. That is why Radix Sort requires a **stable** inner sorting method.

3.3 Complexity

If d is the number of digits and n is the number of items:

$$O(d(n + k))$$

For decimal numbers, $k = 10$, so performance is often close to linear for many practical cases.

Daily-Life / Industry Use

Daily-life analogy: Suppose you arrange exam roll numbers first by last digit, then second last digit, and then third last digit. Finally, the full numbers become ordered.

Industry use:

- Sorting large sets of fixed-length integers
- Postal code processing
- ID number sorting
- Large-scale data systems with digit-based keys

3.4 Implementation Steps

1. Find the largest number.

2. Set digit place = 1.
3. Use Counting Sort on the current digit.
4. Multiply digit place by 10.
5. Repeat until all digits are processed.

3.5 Simple Python Program

```
1 # counting_sort_by_digit() sorts arr according to a specific digit place.
2 def counting_sort_by_digit(arr, place):
3     # Create an output array with the same size as arr.
4     output = [0] * len(arr)
5
6     # Create a count array of size 10 for decimal digits 0 to 9.
7     count = [0] * 10
8
9     # Count how many numbers have each digit at the given place.
10    for number in arr:
11        digit = (number // place) % 10
12        count[digit] += 1
13
14    # Convert the count array into a prefix-sum array.
15    for i in range(1, 10):
16        count[i] += count[i - 1]
17
18    # Build the output array from right to left
19    # to preserve stability.
20    for i in range(len(arr) - 1, -1, -1):
21        digit = (arr[i] // place) % 10
22        output[count[digit] - 1] = arr[i]
23        count[digit] -= 1
24
25    # Copy the output back into the original list.
26    for i in range(len(arr)):
27        arr[i] = output[i]
28
29
30 # radix_sort() sorts arr using repeated digit-wise counting sort.
31 def radix_sort(arr):
32     # If the list is empty, nothing needs to be done.
33     if not arr:
34         return arr
35
36     # Find the largest number to know how many digit places are needed.
37     max_value = max(arr)
38
39     # Start with the units place.
40     place = 1
41
42     # Continue sorting until the highest digit place is processed.
43     while max_value // place > 0:
44         counting_sort_by_digit(arr, place)
45         place *= 10
46
47
48 # Example input list.
49 numbers = [170, 45, 75, 90, 802, 24, 2, 66]
50
```

```
51 # Sort the list in-place.
52 radix_sort(numbers)
53
54 # Display the final sorted list.
55 print("Sorted array:", numbers)
```

Listing 3: Radix Sort in Python with Detailed Comments

4. Trees: Terminology, Binary Tree Types, and Traversals

Concept

A **tree** is a hierarchical data structure made of nodes connected by edges. Unlike arrays and linked lists, trees represent **parent-child relationships**.

A tree starts with a top node called the **root**. Each node may have zero or more child nodes.

4.1 Important Terminology

- **Node:** Basic element of a tree.
- **Root:** Topmost node.
- **Parent:** Node having one or more children.
- **Child:** Node directly below a parent.
- **Leaf:** Node with no children.
- **Sibling:** Nodes with the same parent.
- **Depth:** Distance from root to a node.
- **Height:** Longest path from node to a leaf.
- **Subtree:** Tree inside a tree.

4.2 Binary Tree

A **binary tree** is a tree in which each node has at most two children:

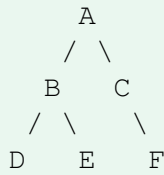
- Left child
- Right child

4.3 Types of Binary Trees

1. **Full Binary Tree:** Every node has either 0 or 2 children.
2. **Complete Binary Tree:** All levels are full except possibly the last, which is filled from left to right.
3. **Perfect Binary Tree:** All internal nodes have 2 children and all leaves are at the same level.
4. **Skewed Tree:** Every node has only one child.
5. **Balanced Tree:** Height remains near $\log n$.

Worked Example

Consider the tree:



Root: A

Leaves: D, E, F

Height of tree: 2 (if root is at level 0)

4.4 Tree Traversals

Traversal means visiting all nodes in a specific order.

4.4.1 Depth-First Traversals

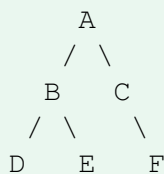
- **Preorder:** Root, Left, Right
- **Inorder:** Left, Root, Right
- **Postorder:** Left, Right, Root

4.4.2 Breadth-First Traversal

- **Level Order:** Visit nodes level by level

Worked Example

For the tree:



Preorder: A, B, D, E, C, F

Inorder: D, B, E, A, C, F

Postorder: D, E, B, F, C, A

Level Order: A, B, C, D, E, F

Algorithm / Working Steps**Inorder Algorithm**

1. Traverse left subtree
2. Visit root
3. Traverse right subtree

Preorder Algorithm

1. Visit root
2. Traverse left subtree
3. Traverse right subtree

Postorder Algorithm

1. Traverse left subtree
2. Traverse right subtree
3. Visit root

Daily-Life / Industry Use**Daily-life analogy:**

- Family hierarchy
- Organization chart
- Folder structure in computer

Industry use:

- Compilers use syntax trees
- Databases use B-Trees
- File systems use directory trees
- AI/game engines use decision trees

4.5 Implementation Steps

1. Create a node class.
2. Each node stores data, left child, and right child.
3. Connect nodes to build the tree.
4. Write traversal functions recursively.

4.6 Simple Python Program

```
1 # Define a class named Node to represent one tree node.
2 class Node:
3     # The constructor runs whenever a new Node object is created.
```

```
4     def __init__(self, value):
5         # Store the data value inside the node.
6         self.value = value
7
8         # Initialize the left child pointer as None,
9         # which means no left child exists yet.
10        self.left = None
11
12        # Initialize the right child pointer as None,
13        # which means no right child exists yet.
14        self.right = None
15
16
17    # inorder() visits nodes in Left, Root, Right order.
18    def inorder(root):
19        # If the current node is None, stop recursion.
20        if root is None:
21            return
22
23        # First visit the left subtree.
24        inorder(root.left)
25
26        # Then print the current node value.
27        print(root.value, end=" ")
28
29        # Finally visit the right subtree.
30        inorder(root.right)
31
32
33    # preorder() visits nodes in Root, Left, Right order.
34    def preorder(root):
35        if root is None:
36            return
37
38        print(root.value, end=" ")
39        preorder(root.left)
40        preorder(root.right)
41
42
43    # postorder() visits nodes in Left, Right, Root order.
44    def postorder(root):
45        if root is None:
46            return
47
48        postorder(root.left)
49        postorder(root.right)
50        print(root.value, end=" ")
51
52
53    # Build a sample binary tree manually.
54    root = Node("A")
55    root.left = Node("B")
56    root.right = Node("C")
57    root.left.left = Node("D")
58    root.left.right = Node("E")
59    root.right.right = Node("F")
60
61    # Display different traversals.
62    print("Inorder traversal:")
63    inorder(root)
```



```
64 print ()
65
66 print ("Preorder traversal:")
67 preorder (root)
68 print ()
69
70 print ("Postorder traversal:")
71 postorder (root)
72 print ()
```

Listing 4: Binary Tree and Traversals in Python with Detailed Comments

5. Binary Search Trees (BSTs): Properties and Core Operations

Concept

A **Binary Search Tree (BST)** is a binary tree with an ordering rule:

- All values in the left subtree are smaller than the node.
- All values in the right subtree are greater than the node.

This property makes searching efficient.

5.1 BST Property

For every node with value x :

$$\text{left subtree values} < x < \text{right subtree values}$$

5.2 Why BST Is Useful

Because the data is ordered, we can search like binary search, but on a tree.

Worked Example

Insert the values:

50, 30, 70, 20, 40, 60, 80

The BST becomes:

50

/ \

30 70

/ \ / \

20 40 60 80

5.3 Core Operations

1. **Search:** Move left if key is smaller, right if larger.
2. **Insert:** Find the correct null position and attach the new node.
3. **Delete:**
 - Node has no child: remove directly.
 - Node has one child: connect parent to child.

- Node has two children: replace with inorder successor or predecessor.

Algorithm / Working Steps

BST Search

1. Start at root.
2. If key equals root, found.
3. If key is smaller, go left.
4. If key is larger, go right.
5. Repeat until found or null.

5.4 Complexity

$$O(h)$$

where h is height of the tree.

- Balanced BST: $O(\log n)$
- Skewed BST: $O(n)$

Daily-Life / Industry Use

Daily-life analogy: Suppose books in a library are arranged by accession number. To find one book, you do not scan every book; you move left or right depending on the number.

Industry use:

- Symbol tables
- Search dictionaries
- Dynamic ordered sets
- Range queries

5.5 Implementation Steps

1. Create BST node class.
2. Write insert function using BST property.
3. Write search function.
4. Write inorder traversal to print values in sorted order.

5.6 Simple Python Program

```
1 # Define a node for the binary search tree.
2 class BSTNode:
3     # The constructor initializes the node with one value.
4     def __init__(self, key):
5         # Store the key in the current node.
6         self.key = key
7
```

```
8         # Left child pointer starts as None.
9         self.left = None
10
11        # Right child pointer starts as None.
12        self.right = None
13
14
15    # insert() adds a new key into the BST.
16    def insert(root, key):
17        # If the current position is empty,
18        # create and return a new node.
19        if root is None:
20            return BSTNode(key)
21
22        # If the new key is smaller than the current node key,
23        # insert it into the left subtree.
24        if key < root.key:
25            root.left = insert(root.left, key)
26
27        # If the new key is greater than the current node key,
28        # insert it into the right subtree.
29        elif key > root.key:
30            root.right = insert(root.right, key)
31
32        # Return the unchanged root pointer.
33        return root
34
35
36    # search() looks for a given key in the BST.
37    def search(root, key):
38        # If the node is None, the key does not exist.
39        if root is None:
40            return False
41
42        # If the current node stores the target key, return True.
43        if root.key == key:
44            return True
45
46        # If the key is smaller, continue in the left subtree.
47        if key < root.key:
48            return search(root.left, key)
49
50        # Otherwise continue in the right subtree.
51        return search(root.right, key)
52
53
54    # inorder() prints the keys in sorted order.
55    def inorder(root):
56        if root is None:
57            return
58
59        inorder(root.left)
60        print(root.key, end=" ")
61        inorder(root.right)
62
63
64    # Create an empty BST.
65    root = None
66
67    # Insert multiple keys.
```

```

68 for value in [50, 30, 70, 20, 40, 60, 80]:
69     root = insert(root, value)
70
71 # Print the BST in sorted order.
72 print("Inorder traversal of BST:")
73 inorder(root)
74 print()
75
76 # Search for a value.
77 print("Is 60 present?", search(root, 60))
78 print("Is 25 present?", search(root, 25))

```

Listing 5: Binary Search Tree in Python with Detailed Comments

6. Heaps: Min/Max Heap Properties, Heapify, and Operations

Concept

A **heap** is a complete binary tree usually stored in an array.

Heap Property

- **Max-Heap:** Parent $\geq$ children
- **Min-Heap:** Parent $\leq$ children

6.1 Why Heap Is Special

A heap is not fully sorted. Only the parent-child relationship is guaranteed. Therefore, the root always gives the largest or smallest element quickly.

6.2 Array Representation

For a node at index i :

$$\text{left child} = 2i + 1, \quad \text{right child} = 2i + 2, \quad \text{parent} = \left\lfloor \frac{i - 1}{2} \right\rfloor$$

6.3 Main Operations

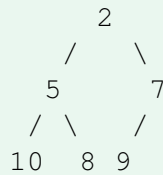
1. **Insert:** Add at end, then move upward (*heapify up*).
2. **Extract Min / Max:** Remove root, move last element to root, then move downward (*heapify down*).
3. **Peek:** Read root without removing it.
4. **Heapify:** Restore heap property.

Worked Example

Min-Heap Example

[2, 5, 7, 10, 8, 9]

This means:



Every parent is smaller than its children.

Daily-Life / Industry Use

Daily-life analogy: Suppose emergency patients are handled in a hospital. The patient with the highest priority must be served first. A heap is ideal for this type of priority processing.

Industry use:

- Priority queues
- CPU scheduling
- Dijkstra's shortest path algorithm
- Event-driven simulation

6.4 Implementation Steps

1. Store heap in a list.
2. Insert at the end.
3. While parent is greater (for min-heap), swap upward.
4. On deletion, move last element to root.
5. Swap downward until heap property is restored.

6.5 Simple Python Program

```

1 # Define a class to implement a simple min-heap.
2 class MinHeap:
3     # The constructor creates an empty list to store heap elements.
4     def __init__(self):
5         self.heap = []
6
7     # insert() adds a new value into the heap.
8     def insert(self, value):
9         # Append the value at the end of the internal list.
10        self.heap.append(value)
11
12        # Restore the heap property by moving the new value upward.
13        self._heapify_up(len(self.heap) - 1)
14
15    # peek() returns the smallest element without removing it.
16    def peek(self):
  
```

```
17     # If the heap is empty, return None.
18     if not self.heap:
19         return None
20
21     # In a min-heap, the root stores the minimum value.
22     return self.heap[0]
23
24     # extract_min() removes and returns the smallest element.
25     def extract_min(self):
26         # If the heap is empty, nothing can be removed.
27         if not self.heap:
28             return None
29
30         # Save the minimum value stored at the root.
31         min_value = self.heap[0]
32
33         # Move the last element to the root position.
34         last_value = self.heap.pop()
35
36         # If the heap still contains elements,
37         # place last_value at the root and heapify downward.
38         if self.heap:
39             self.heap[0] = last_value
40             self._heapify_down(0)
41
42         return min_value
43
44     # _heapify_up() fixes the heap by comparing a node with its parent.
45     def _heapify_up(self, index):
46         # Continue while the node is not the root.
47         while index > 0:
48             # Compute the parent index.
49             parent = (index - 1) // 2
50
51             # If the parent is already smaller or equal, stop.
52             if self.heap[parent] <= self.heap[index]:
53                 break
54
55             # Otherwise swap the child with the parent.
56             self.heap[parent], self.heap[index] = self.heap[index], self.heap[
57                 parent]
58
59             # Move upward to continue checking.
60             index = parent
61
62     # _heapify_down() fixes the heap by comparing a node with its children.
63     def _heapify_down(self, index):
64         size = len(self.heap)
65
66         while True:
67             smallest = index
68             left = 2 * index + 1
69             right = 2 * index + 2
70
71             if left < size and self.heap[left] < self.heap[smallest]:
72                 smallest = left
73
74             if right < size and self.heap[right] < self.heap[smallest]:
75                 smallest = right
```

```

76         if smallest == index:
77             break
78
79         self.heap[index], self.heap[smallest] = self.heap[smallest], self.
            heap[index]
80         index = smallest
81
82
83 # Create a min-heap object.
84 h = MinHeap()
85
86 # Insert values into the heap.
87 for value in [10, 4, 15, 2, 8]:
88     h.insert(value)
89
90 # Show the internal array representation of the heap.
91 print("Heap array:", h.heap)
92
93 # Read the smallest element.
94 print("Minimum element:", h.peek())
95
96 # Remove the smallest element.
97 print("Extracted minimum:", h.extract_min())
98
99 # Show heap after deletion.
100 print("Heap after extraction:", h.heap)

```

Listing 6: Min-Heap in Python with Detailed Comments

7. Balanced Trees (Conceptual): AVL Trees, Red-Black Trees, and B-Trees

Concept

A normal BST can become **skewed**, which makes operations slow. **Balanced trees** keep the height small so that search, insertion, and deletion remain efficient.

7.1 Why Balancing Is Needed

If values are inserted in sorted order into a normal BST:

10, 20, 30, 40, 50

the tree becomes like a linked list:

$O(n)$

search time.

Balanced trees maintain height near:

$O(\log n)$

7.2 AVL Tree

- Strictly balanced
- For every node:

balance factor = height(left) – height(right)

- Allowed values: $-1, 0, 1$
- If balance factor goes out of range, rotate the tree.

7.2.1 AVL Rotations

- **LL Rotation** (single right rotation)
- **RR Rotation** (single left rotation)
- **LR Rotation** (left then right)
- **RL Rotation** (right then left)

7.3 Red-Black Tree

- Each node is red or black.
- Root is black.
- Red node cannot have red child.
- Every root-to-leaf path has same number of black nodes.

It is less strictly balanced than AVL, but insertion/deletion is often efficient in practice.

7.4 B-Tree

- Multi-way search tree
- A node can have multiple keys and children
- Used in disks and databases because it reduces disk access

Feature	AVL Tree	Red-Black Tree	B-Tree
Balance Style	Very strict	Less strict	Multi-way balanced
Search Speed	Very fast	Fast	Fast for disks
Update Cost	Higher	Moderate	Good for external memory
Common Use	Memory-based search	Libraries, maps	Databases, file systems

Daily-Life / Industry Use

Daily-life analogy: Imagine organizing students in rows. If one side becomes too crowded, you redistribute students to keep the arrangement balanced. Tree rotations do the same.

Industry use:

- AVL: in-memory search-heavy applications
- Red-Black: language libraries like ordered maps and sets
- B-Tree: databases, file systems, indexing engines

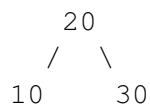
7.5 Step-by-Step AVL Insertion Example

Insert:

30, 20, 10

1. Insert 30 → okay
2. Insert 20 → okay
3. Insert 10 → left-left imbalance at 30
4. Perform right rotation on 30

Final AVL tree:



7.6 Implementation Steps

1. Insert like BST.
2. Update height of nodes.
3. Compute balance factor.
4. If imbalance occurs, apply the correct rotation.

7.7 Simple Python Program (AVL Tree Insertion)

```
1 # Define a node class for the AVL tree.
2 class AVLNode:
3     # Create a node with a key value.
4     def __init__(self, key):
5         self.key = key
6         self.left = None
7         self.right = None
8
9         # Height of a leaf node is initialized as 1.
10        self.height = 1
11
12
13 # height() returns the height of a node.
14 def height(node):
15     if node is None:
16         return 0
17     return node.height
18
19
20 # get_balance() computes the balance factor of a node.
21 def get_balance(node):
22     if node is None:
23         return 0
24     return height(node.left) - height(node.right)
25
26
27 # right_rotate() performs a right rotation.
28 def right_rotate(y):
29     x = y.left
```

```
30     t2 = x.right
31
32     x.right = y
33     y.left = t2
34
35     y.height = 1 + max(height(y.left), height(y.right))
36     x.height = 1 + max(height(x.left), height(x.right))
37
38     return x
39
40
41 # left_rotate() performs a left rotation.
42 def left_rotate(x):
43     y = x.right
44     t2 = y.left
45
46     y.left = x
47     x.right = t2
48
49     x.height = 1 + max(height(x.left), height(x.right))
50     y.height = 1 + max(height(y.left), height(y.right))
51
52     return y
53
54
55 # insert_avl() inserts a key and rebalances the tree if needed.
56 def insert_avl(root, key):
57     # Insert exactly like a BST first.
58     if root is None:
59         return AVLNode(key)
60
61     if key < root.key:
62         root.left = insert_avl(root.left, key)
63     else:
64         root.right = insert_avl(root.right, key)
65
66     # Update the current node height.
67     root.height = 1 + max(height(root.left), height(root.right))
68
69     # Compute balance factor.
70     balance = get_balance(root)
71
72     # Case 1: Left Left
73     if balance > 1 and key < root.left.key:
74         return right_rotate(root)
75
76     # Case 2: Right Right
77     if balance < -1 and key > root.right.key:
78         return left_rotate(root)
79
80     # Case 3: Left Right
81     if balance > 1 and key > root.left.key:
82         root.left = left_rotate(root.left)
83         return right_rotate(root)
84
85     # Case 4: Right Left
86     if balance < -1 and key < root.right.key:
87         root.right = right_rotate(root.right)
88         return left_rotate(root)
89
```

```
90     return root
91
92
93 # inorder() prints keys in sorted order.
94 def inorder(root):
95     if root is None:
96         return
97     inorder(root.left)
98     print(root.key, end=" ")
99     inorder(root.right)
100
101
102 # Build an AVL tree.
103 root = None
104 for value in [30, 20, 10, 25, 40, 50]:
105     root = insert_avl(root, value)
106
107 print("Inorder traversal of AVL tree:")
108 inorder(root)
109 print()
```

Listing 7: AVL Tree Insertion in Python with Detailed Comments

8. Graphs: Terminology and Representations

Concept

A **graph** is a data structure consisting of:

- **Vertices (nodes)**
- **Edges (connections)**

Graphs are used to model networks, roads, friendships, web pages, and communication systems.

8.1 Terminology

- **Vertex:** A node
- **Edge:** Connection between vertices
- **Directed graph:** Edge has direction
- **Undirected graph:** Edge has no direction
- **Weighted graph:** Edges have cost or weight
- **Degree:** Number of edges connected to a vertex
- **Path:** Sequence of connected vertices
- **Cycle:** Path that returns to start

8.2 Graph Representations

8.2.1 Adjacency Matrix

A 2D matrix where:

$$\text{matrix}[i][j] = 1 \text{ if edge exists, else } 0$$

Good for: dense graphs

8.2.2 Adjacency List

For each vertex, store a list of its neighbors.

Good for: sparse graphs

Worked Example

Graph:

A -- B
| |
C -- D

Adjacency List

A: B, C
B: A, D
C: A, D
D: B, C

Feature	Adjacency Matrix	Adjacency List
Memory	High	Low
Edge Check	Fast	Slower
Best For	Dense graphs	Sparse graphs
Traversal Neighbors	Slower	Efficient

Daily-Life / Industry Use

Daily-life analogy:

- People connected in social network
- Cities connected by roads
- Computers connected in network

Industry use:

- Google Maps
- Social media
- Web crawling
- Communication routing

8.3 Implementation Steps

1. Decide whether graph is directed or undirected.
2. Choose adjacency list or matrix.
3. Add vertices.
4. Add edges.

8.4 Simple Python Program

```
1 # Create a graph using an adjacency list.
2 graph = {
3     "A": ["B", "C"],
4     "B": ["A", "D"],
5     "C": ["A", "D"],
6     "D": ["B", "C"]
7 }
8
9 # Print the adjacency list representation.
10 print("Adjacency List Representation:")
11 for vertex, neighbors in graph.items():
12     print(vertex, "->", neighbors)
13
14 # Create the same graph using an adjacency matrix.
15 vertices = ["A", "B", "C", "D"]
16
17 matrix = [
18     [0, 1, 1, 0], # Connections from A
19     [1, 0, 0, 1], # Connections from B
20     [1, 0, 0, 1], # Connections from C
21     [0, 1, 1, 0]  # Connections from D
22 ]
23
24 # Print the adjacency matrix.
25 print("\nAdjacency Matrix Representation:")
26 for row in matrix:
27     print(row)
```

Listing 8: Graph Representation in Python with Detailed Comments

9. Graph Traversal: BFS and DFS

Concept

Graph traversal means visiting all reachable vertices in a systematic way. Two important traversal algorithms are:

- **BFS (Breadth-First Search)**
- **DFS (Depth-First Search)**

9.1 BFS

BFS visits vertices level by level. It uses a **queue**.

9.2 DFS

DFS goes as deep as possible before backtracking. It uses a **stack** or recursion.

Worked Example

For the graph:

A: B, C

B: D, E

C: F

D: -

E: F

F: -

Starting from A:

BFS order: A, B, C, D, E, F

DFS order (one possible): A, B, D, E, F, C

Algorithm / Working Steps

BFS Algorithm

1. Put start node in queue.
2. Mark it visited.
3. Remove one node from front of queue.
4. Visit all unvisited neighbors and add them to queue.
5. Repeat until queue is empty.

DFS Algorithm

1. Start at a node.
2. Mark it visited.
3. Visit one unvisited neighbor recursively.
4. Continue until no unvisited neighbor remains.
5. Backtrack.

9.3 Applications

- BFS:
 - shortest path in unweighted graph
 - web crawling
 - network broadcasting
- DFS:
 - cycle detection
 - topological sort
 - connected components
 - maze solving

Daily-Life / Industry Use

Daily-life analogy:

- BFS: checking all rooms on one floor before going to the next floor
- DFS: entering one corridor deeply, then returning back

9.4 Simple Python Program

```

1 # Import deque because it provides an efficient queue structure.
2 from collections import deque
3
4 # Define the graph using an adjacency list.
5 graph = {
6     "A": ["B", "C"],
7     "B": ["D", "E"],
8     "C": ["F"],
9     "D": [],
10    "E": ["F"],
11    "F": []
12 }
13
14 # bfs() performs Breadth-First Search.
15 def bfs(graph, start):
16     # Create an empty set to remember visited nodes.
17     visited = set()
18
19     # Create a queue and place the start node in it.
20     queue = deque([start])
21
22     # Mark the start node as visited.
23     visited.add(start)
24
25     # Process nodes until the queue becomes empty.
26     while queue:
27         # Remove the leftmost node from the queue.
28         node = queue.popleft()
29
30         # Print the visited node.
31         print(node, end=" ")
32
33         # Explore all neighbors of the current node.
34         for neighbor in graph[node]:
35             # If the neighbor has not been visited,
36             # mark it and add it to the queue.
37             if neighbor not in visited:
38                 visited.add(neighbor)
39                 queue.append(neighbor)
40
41
42 # dfs() performs Depth-First Search using recursion.
43 def dfs(graph, node, visited=None):
44     # If visited is not provided, create an empty set.
45     if visited is None:
46         visited = set()
47
48     # Mark the current node as visited.
49     visited.add(node)

```

```
50     # Print the current node.
51     print(node, end=" ")
52
53     # Visit every unvisited neighbor recursively.
54     for neighbor in graph[node]:
55         if neighbor not in visited:
56             dfs(graph, neighbor, visited)
57
58
59
60 # Run BFS.
61 print("BFS traversal:")
62 bfs(graph, "A")
63 print()
64
65 # Run DFS.
66 print("DFS traversal:")
67 dfs(graph, "A")
68 print()
```

Listing 9: BFS and DFS in Python with Detailed Comments

10. Hashing: Concept, Hash Function, and Collision Resolution

Concept

Hashing is a technique used to store and retrieve data quickly using a **hash function**. A hash function converts a key into an array index:

$$\text{index} = h(\text{key})$$

10.1 Main Idea

Instead of searching linearly, we directly jump to an index computed from the key.

10.2 Hash Table

A **hash table** stores key-value pairs:

(key, value)

Examples:

- student roll number → student record
- username → password metadata
- word → frequency count

10.3 What Makes a Good Hash Function?

A good hash function should:

- distribute keys evenly
- be fast to compute
- reduce collisions

10.4 Collision

A collision occurs when two different keys map to the same index.

10.5 Collision Resolution Techniques

1. **Separate Chaining:** Store a list at each index.
2. **Linear Probing:** Search next free slot.
3. **Quadratic Probing:** Jump using quadratic gaps.
4. **Double Hashing:** Use a second hash function.

Worked Example

Suppose table size = 5 and:

$$h(k) = k \bmod 5$$

Insert:

10, 15, 20

Then:

$$10 \rightarrow 0, \quad 15 \rightarrow 0, \quad 20 \rightarrow 0$$

All three collide at index 0. Using chaining:

index 0 $\rightarrow$ [10, 15, 20]

Daily-Life / Industry Use

Daily-life analogy: Imagine hostel rooms assigned by roll number modulo building size. Sometimes two students get the same room number pattern. Then we need an overflow strategy.

Industry use:

- Dictionaries and maps
- Databases
- Caches
- Compilers' symbol tables
- Password and indexing systems

10.6 Complexity

Average case for search/insert/delete:

$$O(1)$$

Worst case can become:

$$O(n)$$

if many collisions occur.

10.7 Simple Python Program

```
1 # Define a simple hash table class using separate chaining.
2 class HashTable:
3     # The constructor creates the table with a fixed size.
4     def __init__(self, size):
5         self.size = size
6
7         # Create a list of empty lists.
8         # Each list will store all key-value pairs that hash to that index.
9         self.table = [[] for _ in range(size)]
10
11     # _hash() converts a key into an index.
12     def _hash(self, key):
13         # Use Python's built-in hash function,
14         # then limit the index to table size using modulo.
15         return hash(key) % self.size
16
17     # insert() adds or updates a key-value pair.
18     def insert(self, key, value):
19         index = self._hash(key)
20         bucket = self.table[index]
21
22         # If the key already exists, update its value.
23         for i, (existing_key, _) in enumerate(bucket):
24             if existing_key == key:
25                 bucket[i] = (key, value)
26                 return
27
28         # Otherwise append a new pair to the bucket.
29         bucket.append((key, value))
30
31     # search() returns the value associated with a key.
32     def search(self, key):
33         index = self._hash(key)
34         bucket = self.table[index]
35
36         # Look through the bucket to find the key.
37         for existing_key, value in bucket:
38             if existing_key == key:
39                 return value
40
41         # Return None if the key is not present.
42         return None
43
44     # delete() removes a key-value pair if present.
45     def delete(self, key):
46         index = self._hash(key)
47         bucket = self.table[index]
48
49         for i, (existing_key, _) in enumerate(bucket):
50             if existing_key == key:
51                 del bucket[i]
52                 return True
53
54         return False
55
56 # Create a hash table object with 5 buckets.
57 ht = HashTable(5)
58
59
```

```
60 # Insert some data.
61 ht.insert("name", "Aman")
62 ht.insert("city", "Delhi")
63 ht.insert("course", "Data Structures")
64
65 # Search for keys.
66 print("Name:", ht.search("name"))
67 print("City:", ht.search("city"))
68
69 # Delete a key.
70 print("Deleted 'city'?", ht.delete("city"))
71
72 # Check again after deletion.
73 print("City after deletion:", ht.search("city"))
```

Listing 10: Hash Table with Chaining in Python with Detailed Comments

11. Tries (Prefix Trees): Concept and Applications

Concept

A **Trie** is a tree-like data structure used to store strings efficiently. Each node represents one character.

It is especially useful when we care about **prefixes**.

11.1 Main Idea

To store words:

cat, car, care

the common prefix ca is stored only once.

11.2 Structure

Each node usually contains:

- children mapping
- flag to mark end of a word

Worked Example

Words: to, tea, ten

Trie path:

```
root
  t
    o (end)
    e
      a (end)
      n (end)
```

11.3 Operations

1. **Insert(word)**
2. **Search(word)**

3. StartsWith(prefix)

Daily-Life / Industry Use

Daily-life analogy: When you type a word on a mobile keyboard and suggestions appear after only a few letters, a Trie-like structure is often used.

Industry use:

- Auto-complete
- Spell checking
- Search suggestions
- Dictionary applications
- IP routing (prefix matching)

11.4 Implementation Steps

1. Create root node.
2. For each character in word, move/create child node.
3. Mark final node as word-ending.

11.5 Simple Python Program

```
1 # Define a Trie node.
2 class TrieNode:
3     # The constructor creates an empty dictionary for children
4     # and a Boolean flag to mark the end of a complete word.
5     def __init__(self):
6         self.children = {}
7         self.is_end = False
8
9
10 # Define the Trie class.
11 class Trie:
12     # Create the root node.
13     def __init__(self):
14         self.root = TrieNode()
15
16     # insert() adds a word into the trie.
17     def insert(self, word):
18         # Start from the root node.
19         current = self.root
20
21         # Process each character one by one.
22         for ch in word:
23             # If the character is not present,
24             # create a new child node.
25             if ch not in current.children:
26                 current.children[ch] = TrieNode()
27
28             # Move to the next node.
29             current = current.children[ch]
30
31         # Mark the last node as the end of a full word.
```

```

32     current.is_end = True
33
34     # search() returns True if the exact word exists.
35     def search(self, word):
36         current = self.root
37
38         for ch in word:
39             if ch not in current.children:
40                 return False
41             current = current.children[ch]
42
43         return current.is_end
44
45     # starts_with() returns True if any word starts with the given prefix.
46     def starts_with(self, prefix):
47         current = self.root
48
49         for ch in prefix:
50             if ch not in current.children:
51                 return False
52             current = current.children[ch]
53
54         return True
55
56
57 # Create a trie object.
58 trie = Trie()
59
60 # Insert some words.
61 trie.insert("cat")
62 trie.insert("car")
63 trie.insert("care")
64
65 # Search for complete words.
66 print("Search 'cat':", trie.search("cat"))
67 print("Search 'cap':", trie.search("cap"))
68
69 # Search for prefixes.
70 print("Starts with 'ca':", trie.starts_with("ca"))
71 print("Starts with 'do':", trie.starts_with("do"))

```

Listing 11: Trie in Python with Detailed Comments

12. Bloom Filters

Concept

A **Bloom Filter** is a space-efficient probabilistic data structure used to test whether an element is **possibly present** or **definitely absent**.

Important property:

- False positives are possible
- False negatives are not possible

12.1 Main Idea

A Bloom Filter uses:

- a bit array
- multiple hash functions

To add an item:

1. Compute several hash values.
2. Set those bit positions to 1.

To test an item:

1. Compute the same hash values.
2. If any corresponding bit is 0, the item is definitely absent.
3. If all are 1, the item may be present.

Worked Example

Suppose bit array size = 10, and we insert:

"cat "

The three hash functions may produce:

1, 4, 8

So bits 1, 4, and 8 become 1.

If later "car" also produces bits 1, 4, 8, the filter says it may exist, even if it was never inserted. That is a **false positive**.

Daily-Life / Industry Use

Daily-life analogy: Suppose hostel security has a quick rough-check list. If the name is definitely not in the rough system, entry is denied immediately. If it seems present, they check the main register.

Industry use:

- Databases
- Web caching
- Distributed systems
- Spell check prefiltering
- Blockchain and storage systems

12.2 Implementation Steps

1. Create a bit array of fixed size.
2. Define multiple hash functions.
3. For add operation, set relevant bits to 1.
4. For search operation, test those bits.

12.3 Simple Python Program

```
1 # Import hashlib to create reproducible hash values.
2 import hashlib
3
4 # Define a simple Bloom Filter class.
5 class BloomFilter:
6     # The constructor creates the bit array and stores the number of hash
7     # functions.
8     def __init__(self, size, hash_count):
9         self.size = size
10        self.hash_count = hash_count
11
12        # Create the bit array with all bits initially set to 0.
13        self.bit_array = [0] * size
14
15        # _hashes() generates multiple hash values for one item.
16        def _hashes(self, item):
17            positions = []
18
19            # Use different salt values to simulate multiple hash functions.
20            for i in range(self.hash_count):
21                # Create a new string by attaching i to the item.
22                data = f"{item}-{i}".encode()
23
24                # Compute the hash digest and convert it to an integer.
25                digest = int(hashlib.md5(data).hexdigest(), 16)
26
27                # Map the large integer into the bit array range.
28                positions.append(digest % self.size)
29
30            return positions
31
32        # add() inserts an item into the Bloom Filter.
33        def add(self, item):
34            for position in self._hashes(item):
35                self.bit_array[position] = 1
36
37        # contains() checks whether the item may be present.
38        def contains(self, item):
39            for position in self._hashes(item):
40                if self.bit_array[position] == 0:
41                    return False
42            return True
43
44        # Create a Bloom Filter object.
45        bf = BloomFilter(size=20, hash_count=3)
46
47        # Add some items.
48        bf.add("cat")
49        bf.add("dog")
50        bf.add("fish")
51
52        # Query the filter.
53        print("Is 'cat' maybe present?", bf.contains("cat"))
54        print("Is 'bird' maybe present?", bf.contains("bird"))
```

Listing 12: Bloom Filter in Python with Detailed Comments

13. Spatial Data Structures: Quadrees and K-D Trees

Concept

Spatial data structures organize data points in geometric space, such as 2D or 3D coordinates. They are used when data is based on location, area, region, or dimension.

13.1 Why Spatial Structures Are Needed

If we store points as a normal list, searching nearby points becomes slow. Spatial structures allow efficient region search, nearest-neighbor search, and partitioning.

13.2 Quadrees

A **Quadtree** divides 2D space into four quadrants repeatedly:

- North-West
- North-East
- South-West
- South-East

Useful for 2D maps, image segmentation, and game collision detection.

13.3 K-D Trees

A **K-D Tree** (*k-dimensional tree*) stores points in k -dimensional space. It alternates splitting by dimensions:

- level 0: split by x
- level 1: split by y
- level 2: split by x
- and so on

Worked Example

Points:

$(3, 6), (17, 15), (13, 15), (6, 12), (9, 1), (2, 7), (10, 19)$

A K-D Tree stores these points while alternating comparisons on x and y . This supports efficient range and nearest-neighbor queries.

Daily-Life / Industry Use**Daily-life analogy:**

- Quadtree: dividing a city map into four regions, then subregions
- K-D Tree: finding nearest food delivery location or nearest hospital

Industry use:

- GIS and maps
- Image processing
- Robotics
- CAD tools
- Game engines

13.4 Implementation Steps for K-D Tree

1. Choose one axis based on depth.
2. Compare point by that axis.
3. Insert left if smaller, right if larger.
4. Alternate axis at each level.

13.5 Simple Python Program (K-D Tree)

```
1 # Define a node class for storing one 2D point.
2 class KNode:
3     # The constructor stores the point and child pointers.
4     def __init__(self, point):
5         self.point = point
6         self.left = None
7         self.right = None
8
9
10 # insert_kd() inserts a point into a 2D K-D Tree.
11 def insert_kd(root, point, depth=0):
12     # If the current root is empty, create a new node.
13     if root is None:
14         return KNode(point)
15
16     # Decide which axis to compare:
17     # axis 0 means x-coordinate, axis 1 means y-coordinate.
18     axis = depth % 2
19
20     # Compare the point on the selected axis.
21     if point[axis] < root.point[axis]:
22         root.left = insert_kd(root.left, point, depth + 1)
23     else:
24         root.right = insert_kd(root.right, point, depth + 1)
25
26     return root
27
28
```

```

29 # preorder_kd() prints the K-D Tree in preorder form.
30 def preorder_kd(root):
31     if root is None:
32         return
33     print(root.point, end=" ")
34     preorder_kd(root.left)
35     preorder_kd(root.right)
36
37
38 # Create an empty K-D Tree.
39 root = None
40
41 # Insert some 2D points.
42 points = [(3, 6), (17, 15), (13, 15), (6, 12), (9, 1), (2, 7), (10, 19)]
43 for point in points:
44     root = insert_kd(root, point)
45
46 # Print the tree.
47 print("Preorder traversal of K-D Tree:")
48 preorder_kd(root)
49 print()

```

Listing 13: K-D Tree in Python with Detailed Comments

13.6 Mini Python Program (Quadtree Conceptual Skeleton)

```

1 # Define a rectangle region using x, y, width, and height.
2 class Rectangle:
3     def __init__(self, x, y, w, h):
4         self.x = x        # x-coordinate of top-left corner
5         self.y = y        # y-coordinate of top-left corner
6         self.w = w        # width of the rectangle
7         self.h = h        # height of the rectangle
8
9     # contains() checks whether a point lies inside the rectangle.
10    def contains(self, point):
11        px, py = point
12        return self.x <= px < self.x + self.w and self.y <= py < self.y + self.
13           h
14
15 # Define a simple Quadtree node.
16 class Quadtree:
17     def __init__(self, boundary, capacity):
18         self.boundary = boundary    # Region covered by this node
19         self.capacity = capacity    # Maximum points before subdivision
20         self.points = []            # Points stored in this region
21         self.divided = False        # Whether subdivision happened
22
23     # insert() adds a point if it lies inside the boundary.
24     def insert(self, point):
25         if not self.boundary.contains(point):
26             return False
27
28         if len(self.points) < self.capacity:
29             self.points.append(point)
30             return True
31
32     # In a full implementation, we would subdivide into

```

```
33         # NW, NE, SW, and SE child regions here.
34         self.divided = True
35         return False
36
37
38 # Create one region and insert points conceptually.
39 boundary = Rectangle(0, 0, 100, 100)
40 qt = Quadtree(boundary, 2)
41
42 print(qt.insert((10, 20)))
43 print(qt.insert((30, 40)))
44 print(qt.insert((70, 80))) # This would trigger subdivision in a full Quadtree
45 print("Stored points:", qt.points)
46 print("Subdivision happened?", qt.divided)
```

Listing 14: Very Simple Quadtree Skeleton in Python with Detailed Comments

14. Fenwick Tree and Segment Tree

Concept

Both **Fenwick Tree** and **Segment Tree** help answer range queries efficiently.
Examples of range queries:

- sum from index l to r
- minimum in a range
- number of updates and prefix totals

14.1 Fenwick Tree (Binary Indexed Tree)

Fenwick Tree is simpler and mainly used for:

- prefix sums
- point updates

14.1.1 Main Idea

Store cumulative information in a clever binary-indexed way.

14.2 Segment Tree

Segment Tree is more general:

- range sum
- range minimum
- range maximum
- range updates (with advanced versions)

Feature	Fenwick Tree	Segment Tree
Implementation	Simple	More complex
Memory	Less	More
Prefix Sum	Excellent	Good
General Range Queries	Limited	Very flexible

Worked Example

Array:

[1, 2, 3, 4, 5]

Query:

$$\text{sum}(2, 4) = 2 + 3 + 4 = 9$$

A normal loop takes $O(n)$. Fenwick Tree and Segment Tree can answer much faster after preprocessing.

Daily-Life / Industry Use

Daily-life analogy: Imagine a class attendance system where the teacher repeatedly asks:

- how many students attended from roll number 1 to 20?
- then 1 to 50?
- then 21 to 40?

A range-query structure saves time compared to recounting every time.

Industry use:

- competitive programming
- financial cumulative totals
- analytics dashboards
- game score aggregation

14.3 Fenwick Tree Implementation Steps

1. Create an array of size $n + 1$.
2. Use index tricks based on least significant set bit.
3. Update affects multiple cumulative nodes.
4. Prefix sum jumps backward using bit operations.

14.4 Simple Python Program (Fenwick Tree)

```

1 # Define a Fenwick Tree class.
2 class FenwickTree:
3     # The constructor creates an internal tree array of size n + 1.
4     def __init__(self, n):
5         self.n = n
6         self.tree = [0] * (n + 1)
7 
```

```

8      # update() adds delta to one index.
9      def update(self, index, delta):
10         # Fenwick Tree uses 1-based indexing internally.
11         while index <= self.n:
12             self.tree[index] += delta
13
14             # Move to the next responsible index.
15             index += index & -index
16
17     # prefix_sum() returns the sum from 1 to index.
18     def prefix_sum(self, index):
19         total = 0
20         while index > 0:
21             total += self.tree[index]
22
23             # Move to the parent contribution index.
24             index -= index & -index
25         return total
26
27     # range_sum() returns the sum from left to right.
28     def range_sum(self, left, right):
29         return self.prefix_sum(right) - self.prefix_sum(left - 1)
30
31
32     # Create a Fenwick Tree for 5 elements.
33     ft = FenwickTree(5)
34
35     # Original array values: [1, 2, 3, 4, 5]
36     values = [1, 2, 3, 4, 5]
37
38     # Build the tree by updating each position.
39     for i, value in enumerate(values, start=1):
40         ft.update(i, value)
41
42     # Query sums.
43     print("Prefix sum up to index 3:", ft.prefix_sum(3))
44     print("Range sum from index 2 to 4:", ft.range_sum(2, 4))

```

Listing 15: Fenwick Tree in Python with Detailed Comments

14.5 Simple Python Program (Segment Tree)

```

1  # Define a Segment Tree class for range-sum queries.
2  class SegmentTree:
3      # The constructor builds the tree from an input array.
4      def __init__(self, arr):
5          self.n = len(arr)
6
7          # Allocate enough space for the segment tree.
8          self.tree = [0] * (4 * self.n)
9
10         # Build the segment tree starting from the root.
11         self._build(arr, 1, 0, self.n - 1)
12
13     # _build() constructs the tree recursively.
14     def _build(self, arr, node, left, right):
15         # If the segment contains only one element,
16         # store that value directly.
17         if left == right:

```

```

18         self.tree[node] = arr[left]
19         return
20
21     mid = (left + right) // 2
22
23     # Build left and right child segments.
24     self._build(arr, 2 * node, left, mid)
25     self._build(arr, 2 * node + 1, mid + 1, right)
26
27     # Store the sum of both children at the current node.
28     self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]
29
30     # query() returns the sum in the range [ql, qr].
31     def query(self, ql, qr):
32         return self._query(1, 0, self.n - 1, ql, qr)
33
34     # _query() works recursively on the segment tree.
35     def _query(self, node, left, right, ql, qr):
36         # If the current segment is completely outside the query range,
37         # contribute 0.
38         if qr < left or right < ql:
39             return 0
40
41         # If the current segment lies completely inside the query range,
42         # return its stored sum.
43         if ql <= left and right <= qr:
44             return self.tree[node]
45
46         mid = (left + right) // 2
47
48         # Otherwise split into left and right children.
49         return self._query(2 * node, left, mid, ql, qr) + \
50             self._query(2 * node + 1, mid + 1, right, ql, qr)
51
52
53     # Build a segment tree from an array.
54     arr = [1, 2, 3, 4, 5]
55     st = SegmentTree(arr)
56
57     # Query the sum from index 1 to 3.
58     print("Range sum from index 1 to 3:", st.query(1, 3))

```

Listing 16: Segment Tree in Python with Detailed Comments

15. Skip Lists

Concept

A **Skip List** is a probabilistic data structure that stores sorted elements and supports fast search, insertion, and deletion. It can be seen as a layered linked list.

15.1 Main Idea

A normal sorted linked list allows easy insertion but slow search.

A skip list adds **express lanes** at higher levels:

- bottom level contains all elements

- upper levels contain selected shortcut links

This makes search much faster on average.

Worked Example

Suppose bottom level is:

$$1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 9 \rightarrow 11$$

A higher level may skip some nodes:

$$1 \rightarrow 5 \rightarrow 9$$

Search first moves fast on upper levels, then descends.

15.2 Why Skip Lists Are Interesting

Skip lists often provide average:

$$O(\log n)$$

for search, insertion, and deletion, while remaining easier to implement than some balanced trees.

Daily-Life / Industry Use

Daily-life analogy: Imagine a multi-floor library where:

- Ground floor shelves contain all books.
- Upper floors have shortcut indexes to major sections.

You use shortcuts first, then move down for exact search.

Industry use:

- databases
- key-value stores
- in-memory ordered collections
- systems where simple balanced performance is needed

15.3 Implementation Steps

1. Create node with multiple forward pointers.
2. Randomly decide the level of a new node.
3. While searching, move right while possible, otherwise move down.

15.4 Simple Python Program

```

1 # Import the random module to generate random levels.
2 import random
3
4 # Set a maximum number of levels for the skip list.
5 MAX_LEVEL = 4
6
7 # Define a node for the skip list.
8 class SkipNode:
9     # The constructor stores the key and creates forward pointers.
10     def __init__(self, key, level):

```

```
11         self.key = key
12         self.forward = [None] * (level + 1)
13
14
15 # Define the Skip List class.
16 class SkipList:
17     # Create the header node and initial level.
18     def __init__(self):
19         self.level = 0
20         self.header = SkipNode(-1, MAX_LEVEL)
21
22     # random_level() decides how many levels a new node gets.
23     def random_level(self):
24         level = 0
25         while random.random() < 0.5 and level < MAX_LEVEL:
26             level += 1
27         return level
28
29     # insert() adds a key into the skip list.
30     def insert(self, key):
31         update = [None] * (MAX_LEVEL + 1)
32         current = self.header
33
34         # Start from the highest current level and move downward.
35         for i in range(self.level, -1, -1):
36             while current.forward[i] and current.forward[i].key < key:
37                 current = current.forward[i]
38             update[i] = current
39
40         current = current.forward[0]
41
42         # Insert only if the key is not already present.
43         if current is None or current.key != key:
44             random_lvl = self.random_level()
45
46             if random_lvl > self.level:
47                 for i in range(self.level + 1, random_lvl + 1):
48                     update[i] = self.header
49                 self.level = random_lvl
50
51             new_node = SkipNode(key, random_lvl)
52
53             for i in range(random_lvl + 1):
54                 new_node.forward[i] = update[i].forward[i]
55                 update[i].forward[i] = new_node
56
57     # search() checks whether a key exists.
58     def search(self, key):
59         current = self.header
60
61         for i in range(self.level, -1, -1):
62             while current.forward[i] and current.forward[i].key < key:
63                 current = current.forward[i]
64
65         current = current.forward[0]
66         return current is not None and current.key == key
67
68     # display() prints the bottom level of the skip list.
69     def display(self):
70         current = self.header.forward[0]
```



```
71     while current:
72         print(current.key, end=" ")
73         current = current.forward[0]
74     print()
75
76
77 # Create a skip list object.
78 sl = SkipList()
79
80 # Insert sorted values.
81 for value in [3, 6, 7, 9, 12, 19]:
82     sl.insert(value)
83
84 # Display the bottom level.
85 print("Skip list contents:")
86 sl.display()
87
88 # Search for keys.
89 print("Search 9:", sl.search(9))
90 print("Search 15:", sl.search(15))
```

Listing 17: Skip List in Python with Detailed Comments

16. Quick Revision Tables

16.1 Sorting Summary

Algorithm	Best/Average/Worst	Extra Space	Stable	Type
Heap Sort	$O(n \log n)$	/	No	Comparison
	$O(n \log n)$	$O(1)$		
	$O(n \log n)$			
Counting Sort	$O(n + k)$	$O(n + k)$	Yes	Non-comparison
Radix Sort	$O(d(n + k))$	$O(n + k)$	Yes	Non-comparison

16.2 Tree and Graph Summary

Structure	Key Strength
Tree	Hierarchical representation
BST	Ordered search in dynamic tree
Heap	Fast priority access
AVL / Red-Black / B-Tree	Balanced search performance
Graph	General network modeling
Trie	Efficient prefix search
Bloom Filter	Fast probabilistic membership test
K-D Tree / Quadtree	Efficient spatial queries
Fenwick / Segment Tree	Fast range queries and updates
Skip List	Fast average ordered search with simple structure

17. Common Mistakes Students Must Avoid

Student Tips

1. Do not confuse a **heap** with a **BST**. A heap only guarantees parent-child order, not full left-right order.
2. Counting Sort and Radix Sort are not suitable for every type of data.
3. In BST, insertion order matters a lot.
4. In DFS and tree traversals, always define the **base case**.
5. In hashing, collisions are normal; they must be handled properly.
6. Bloom Filter can say **maybe present**, not always definitely present.
7. Fenwick Tree is simpler than Segment Tree, but less flexible.
8. Skip List uses randomness; different runs may create different levels.

18. Practice Questions for Students

1. Why is Heap Sort not stable?
2. Why must Counting Sort know the value range?
3. Why does Radix Sort require a stable internal sort?
4. Explain the difference between a tree and a graph.
5. Why can a BST become inefficient?
6. Why are heaps useful in priority scheduling?
7. Compare AVL and Red-Black trees.
8. When is adjacency list better than adjacency matrix?
9. Why is BFS suitable for shortest path in unweighted graphs?
10. What is a collision in hashing?
11. Why are tries useful in auto-complete?
12. Why does a Bloom Filter allow false positives?
13. Explain one real-world use of spatial data structures.
14. Compare Fenwick Tree and Segment Tree.
15. Why is Skip List called a probabilistic structure?

Conclusion

These topics form the foundation of **advanced problem solving, system design, efficient searching, fast updates, and real-world data organization**. A student who understands these structures deeply can solve problems more intelligently and can also appreciate how real software systems work internally.